

Estudo Dirigido · Guia 4

Sistema de Colisão

Detecção e Interação entre Elementos do Jogo

Este guia aborda o mecanismo que dá vida aos jogos: a detecção de colisão. Aprenderemos a usar o objeto Rect do Pygame para verificar interações entre sprites — de colisões simples entre dois objetos até sistemas eficientes para grupos inteiros de inimigos e projéteis.

Objetivos de Aprendizagem

- Compreender o objeto `pygame.Rect` e seus atributos de posição
- Criar e atualizar Rects a partir de sprites e posições
- Detectar colisão entre dois Rects com `colliderect()`
- Gerenciar colisões de um sprite contra múltiplos inimigos
- Usar `groupcollide()` para colisões eficientes entre grupos de sprites

1. O Objeto `pygame.Rect`

Em Pygame, todo sprite pode ter um Rect (retângulo) associado que define sua posição e dimensões na tela. O Rect é a ferramenta central para detecção de colisão — verificar a interseção entre retângulos é muito mais rápido e eficiente do que comparar pixels individuais.

Posição	Dimensões	Colisão
<code>x, y, topleftcenter, bottomright</code> Coordenadas do sprite na tela	<code>width, height, size</code> Tamanho em pixels da imagem	<code>colliderect()</code> <code>collidepoint()</code> Detecta sobreposição com outros Rects

1.1. Criando e Posicionando um Rect

Para obter o Rect de um sprite, use o método `get_rect()` da Surface. Em seguida, defina a posição usando um dos atributos de ancoragem disponíveis:

```
# PLAYER_IMAGE é a Surface (imagem) carregada
player_rect = PLAYER_IMAGE.get_rect()

# Opção 1: posicionar pelo canto superior esquerdo (padrão)
```

```
player_rect.topleft = (player_x, player_y)

# Opção 2: posicionar pelo centro
player_rect.center = (player_x, player_y)

# Opção 3: criar já com posição inicial (forma mais concisa)
player_rect = PLAYER_IMAGE.get_rect(topleft=(player_x, player_y))
```

Atributos de Posição do Rect

Atributo	Tipo	Descrição
<code>x, y</code>	<code>int</code>	Canto superior esquerdo (equivale a <code>topleft</code>)
<code>topleft</code>	<code>tupla</code>	Canto superior esquerdo (x, y)
<code>topright</code>	<code>tupla</code>	Canto superior direito
<code>bottomleft</code>	<code>tupla</code>	Canto inferior esquerdo
<code>bottomright</code>	<code>tupla</code>	Canto inferior direito
<code>center</code>	<code>tupla</code>	Centro do retângulo (cx, cy)
<code>centerx/centery</code>	<code>int</code>	Coordenada X ou Y do centro individualmente
<code>width, height</code>	<code>int</code>	Largura e altura do retângulo

⚠ Manter o Rect sincronizado

O Rect NÃO se move automaticamente quando você altera `player_x` e `player_y`. A cada frame, antes de verificar colisões, atualize: `player_rect.topleft = (player_x, player_y)`. Se esquecer, as colisões serão calculadas na posição ANTIGA do sprite.

2. Colisão entre Dois Sprites

2.1. O Método `colliderect()`

O método mais simples para detectar colisão entre dois objetos é `colliderect()`. Ele retorna `True` se os dois Rects se sobrepõem, e `False` caso contrário:

```
# Supondo que temos dois Rects atualizados:
# player_bullet_rect → Rect do projétil do jogador
# enemy_rect          → Rect de um inimigo

if player_bullet_rect.colliderect(enemy_rect):
    print("Colisão! Inimigo atingido.")
    # Lógica de jogo: remover tiro, descontar vida, adicionar pontos...
```

💡 Onde colocar o código de colisão?

Toda a lógica de detecção de colisão deve ficar na seção '2. Atualizar o Estado' do Game Loop, APÓS mover os sprites mas ANTES de renderizá-los. Isso garante que a posição está atualizada quando a colisão é verificada.

2.2. Fluxo Completo: Tiro x Inimigo

O exemplo abaixo mostra o ciclo completo: atualizar o Rect, verificar colisão e aplicar a lógica de jogo:

```
# — Dentro da seção '2. Atualizar o Estado' —————

# 1. Atualiza o Rect do tiro com a posição atual
bullet_rect = BULLET_IMAGE.get_rect(topleft=(bullet_x, bullet_y))

# 2. Atualiza o Rect do inimigo com a posição atual
enemy_rect = ENEMY_IMAGE.get_rect(topleft=(enemy_x, enemy_y))

# 3. Verifica a colisão
if bullet_rect.colliderect(enemy_rect):
    bullet_active = False    # remove o tiro
    enemy_active = False     # remove o inimigo
    score += 10              # incrementa pontuação
```

3. Colisão com Múltiplos Inimigos

Na maioria dos jogos, há dezenas de inimigos e projéteis simultâneos. Verificar colisões um a um com `colliderect()` funciona, mas o Pygame oferece funções especializadas para grupos, que são mais eficientes e organizam melhor o código.

3.1. Abordagem Manual: Loop Aninhado

A forma mais direta — sem usar o módulo `sprite` — é iterar sobre todos os tiros e verificar contra todos os inimigos:

```
# Listas de Rects ativos
bullet_rects = [BULLET_IMAGE.get_rect(topleft=(bx, by))
                 for (bx, by) in bullets_positions]
enemy_rects = [ENEMY_IMAGE.get_rect(topleft=(ex, ey))
               for (ex, ey) in enemies_positions]

bullets_to_remove = []
```

```
enemies_to_remove = []

for i, b_rect in enumerate(bullet_rects):
    for j, e_rect in enumerate(enemy_rects):
        if b_rect.colliderect(e_rect):
            bullets_to_remove.append(i)
            enemies_to_remove.append(j)
            score += 10

# Remove os objetos colididos (de trás pra frente para não bagunçar índices)
for i in sorted(set(bullets_to_remove), reverse=True):
    bullets_positions.pop(i)
for j in sorted(set(enemies_to_remove), reverse=True):
    enemies_positions.pop(j)
```

⚠ Cuidado ao remover durante iteração

Nunca remova elementos de uma lista enquanto itera sobre ela. Colete os índices a remover em uma lista auxiliar e aplique as remoções DEPOIS do loop, de trás pra frente (`reverse=True`) para não deslocar os índices restantes.

3.2. Usando `pygame.sprite.Sprite` e `Group`

A abordagem recomendada para projetos maiores é usar as classes `pygame.sprite.Sprite` e `pygame.sprite.Group`. Elas organizam os objetos do jogo e habilitam as funções de colisão automática do Pygame:

```
import pygame

# — Classe base para qualquer sprite do jogo —————
class GameSprite(pygame.sprite.Sprite):
    def __init__(self, image, x, y):
        super().__init__()
        self.image = image
        self.rect = image.get_rect(topleft=(x, y))

# — Criando grupos de sprites —————
all_bullets = pygame.sprite.Group()    # grupo de projéteis
all_enemies = pygame.sprite.Group()    # grupo de inimigos

# Adicionando um projétil ao grupo
new_bullet = GameSprite(BULLET_IMAGE, bullet_x, bullet_y)
all_bullets.add(new_bullet)

# Adicionando inimigos ao grupo
for i in range(5):
```

```
enemy = GameSprite(ENEMY_IMAGE, 100 * i, 50)
all_enemies.add(enemy)
```

💡 Por que usar Sprite e Group?

Além das funções de colisão automática, os Groups permitem atualizar e desenhar todos os sprites de uma só vez com `group.update()` e `group.draw(SCREEN)`, simplificando muito o Game Loop.

3.3. `groupcollide()`: Grupo contra Grupo

Para verificar colisões entre todos os projéteis e todos os inimigos simultaneamente, use `pygame.sprite.groupcollide()`. É a função mais eficiente para sistemas de combate:

```
# Sintaxe:
# groupcollide(grupo_a, grupo_b, kill_a, kill_b)
#   kill_a → True: remove sprites de grupo_a que colidiram
#   kill_b → True: remove sprites de grupo_b que colidiram
#   Retorno: dicionário {sprite_a: [lista de sprites_b colididos]}

collisions = pygame.sprite.groupcollide(
    all_bullets, # grupo A: projéteis do jogador
    all_enemies, # grupo B: inimigos
    True,        # kill_a: remove o tiro após colidir
    True         # kill_b: remove o inimigo após colidir
)

# Processa as colisões detectadas
for bullet, enemies_hit in collisions.items():
    for enemy in enemies_hit:
        score += enemy.points # ex: atributo points na classe Enemy
        # Aqui: tiro e inimigo já foram removidos dos grupos (kill=True)
        # Adicione efeitos de explosão, som, animação, etc.
```

Comparativo das Abordagens de Colisão

Método	Uso Ideal	Complexidade	Eficiência
<code>colliderect()</code>	1 vs 1 (dois objetos)	Baixa	★★★★
Loop manual	1 vs muitos (sem classes)	Média	★★★
<code>groupcollide()</code>	Muitos vs muitos (com Groups)	Alta (Pygame cuida)	★★★★★★

4. Código Completo: Jogo com Colisão

O exemplo abaixo demonstra um jogo funcional com jogador, inimigo, projétil e detecção de colisão usando Groups:

```
import pygame, sys

# — Classe base —
class GameSprite(pygame.sprite.Sprite):
    def __init__(self, color, width, height, x, y):
        super().__init__()
        self.image = pygame.Surface((width, height))
        self.image.fill(color)
        self.rect = self.image.get_rect(topleft=(x, y))
        self.points = 10

# — Inicialização —
pygame.init()
SCREEN = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Colisão Demo")
clock = pygame.time.Clock()

# — Sprites —
player = GameSprite((0, 180, 255), 50, 40, 375, 530)
enemy = GameSprite((255, 60, 60), 50, 40, 375, 80)

all_bullets = pygame.sprite.Group()
all_enemies = pygame.sprite.Group()
all_enemies.add(enemy)

score = 0
SPEED = 5

# — Game Loop —
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT: running = False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_SPACE:
                # Dispara um projétil a partir do centro do jogador
                b = GameSprite((255, 255, 0), 6, 16,
                               player.rect.centerx - 3, player.rect.top)
                all_bullets.add(b)

    # Movimentação do jogador
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]: player.rect.x -= SPEED
```

```

if keys[pygame.K_RIGHT]: player.rect.x += SPEED

# Move os projéteis para cima
for b in all_bullets:
    b.rect.y -= 8
    if b.rect.bottom < 0: b.kill() # remove ao sair da tela

# Detecção de colisão: tiros vs inimigos
hits = pygame.sprite.groupcollide(all_bullets, all_enemies, True, True)
for bullet, enemies_hit in hits.items():
    for e in enemies_hit:
        score += e.points
        print(f"Inimigo destruído! Pontos: {score}")

# Renderização
SCREEN.fill((0, 0, 20))
all_bullets.draw(SCREEN)
all_enemies.draw(SCREEN)
SCREEN.blit(player.image, player.rect)
pygame.display.flip()
clock.tick(60)

pygame.quit(); sys.exit()

```

5. Referência de Funções de Colisão

Função / Método	Descrição e Retorno
<code>rect.collidect(r2)</code>	True se rect e r2 se sobrepõem
<code>rect.collidepoint((x,y))</code>	True se o ponto (x,y) está dentro do rect
<code>rect.collidelist(lista)</code>	Índice do primeiro Rect da lista que colidiu (-1 se nenhum)
<code>rect.collidelistall(lista)</code>	Lista de todos os índices que colidiram
<code>sprite.kill()</code>	Remove o sprite de TODOS os grupos a que pertence
<code>group.add(sprite)</code>	Adiciona um sprite ao grupo
<code>group.remove(sprite)</code>	Remove um sprite específico do grupo
<code>group.draw(surface)</code>	Desenha todos os sprites do grupo na surface
<code>group.update(*args)</code>	Chama update() em cada sprite do grupo
<code>groupcollide(ga, gb, ka, kb)</code>	Colisão grupo×grupo; retorna dict de colisões

Função / Método	Descrição e Retorno
<code>spritecollideany(s, group)</code>	Retorna qualquer sprite do group que colidiu com s (ou None)
<code>spritecollide(s, group, kill)</code>	Lista de sprites do group que colidiram com s

6. Exercícios Práticos

Consolide os conceitos com os desafios abaixo:

Exercício 1 — Primeiro coliderect()

Crie dois retângulos (usando `pygame.draw.rect`) e detecte quando o mouse entra na área de um deles usando `collidepoint()`.

Exercício 2 — Grade de inimigos

Crie uma grade de 3×3 inimigos e adicione-os a um Group. Use `groupcollide()` para destruir aqueles que forem atingidos pelo tiro.

Exercício 3 — Colisão jogador x inimigo

Detecte a colisão entre o jogador e qualquer inimigo do grupo usando `spritecollideany()`. Ao colidir, exiba 'Game Over' no terminal.

Exercício 4 — `kill_b = False`

Experimente chamar `groupcollide()` com `kill_b=False`. O que muda? Para que serve esse comportamento? (Dica: tiro que atravessa múltiplos inimigos.)

Exercício 5 — Sistema de vidas

Adicione um atributo `health = 3` na classe `Enemy`. Em vez de destruir no primeiro tiro, decamente a vida. Só destrua o inimigo quando `health` chegar a 0.

7. Próximos Passos

Com o sistema de colisão implementado, o jogo já tem interatividade real. Os próximos guias completarão a experiência:

Guia 5 — Sons e Efeitos

- `pygame.mixer` para efeitos sonoros de tiro e explosão
- `pygame.mixer.music` para trilha sonora
- Animações de explosão com spritesheets

Guia 6 — HUD e Estados de Jogo

- `pygame.font` para placar e vidas na tela
- Telas de Game Over e Vitória
- Máquina de estados: menu, jogo, pausa